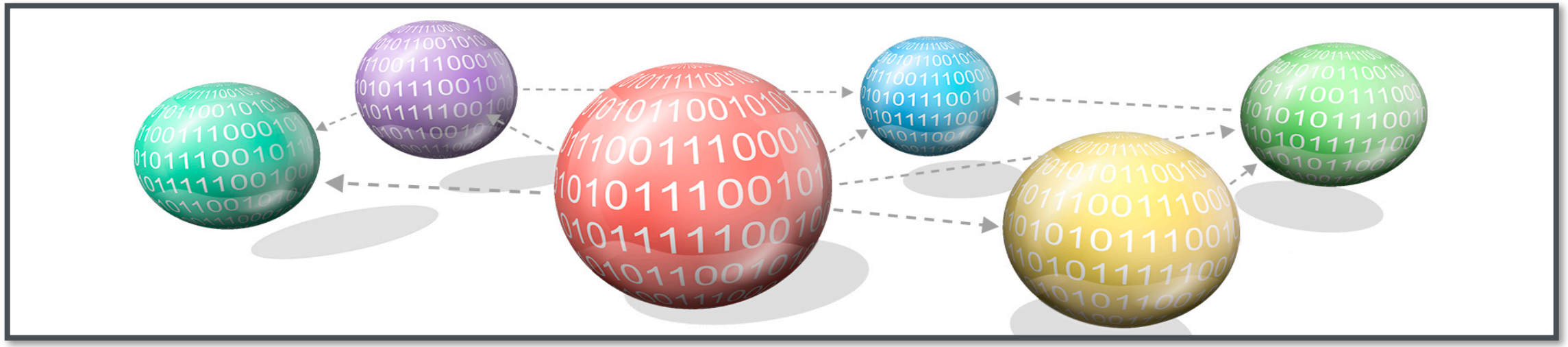
NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

Chapter 5: Polymorphism

CE2002 Object Oriented Design & Programming

Dr Zhang Jie
Assoc Prof, School of Computer Science and Engineering (SCSE)



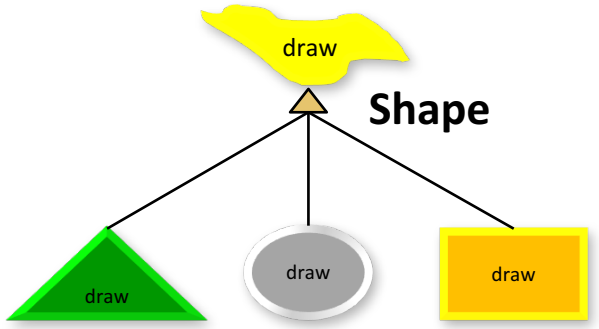
Learning Objectives

After the completion of this chapter, you should be able to:

- Explain polymorphism.
- Explain the concept of binding.
- Describe object typecasting.
- List the benefits of polymorphism.
- Explain the three ways of method overriding.

Topic 1: Polymorphism

Polymorphism



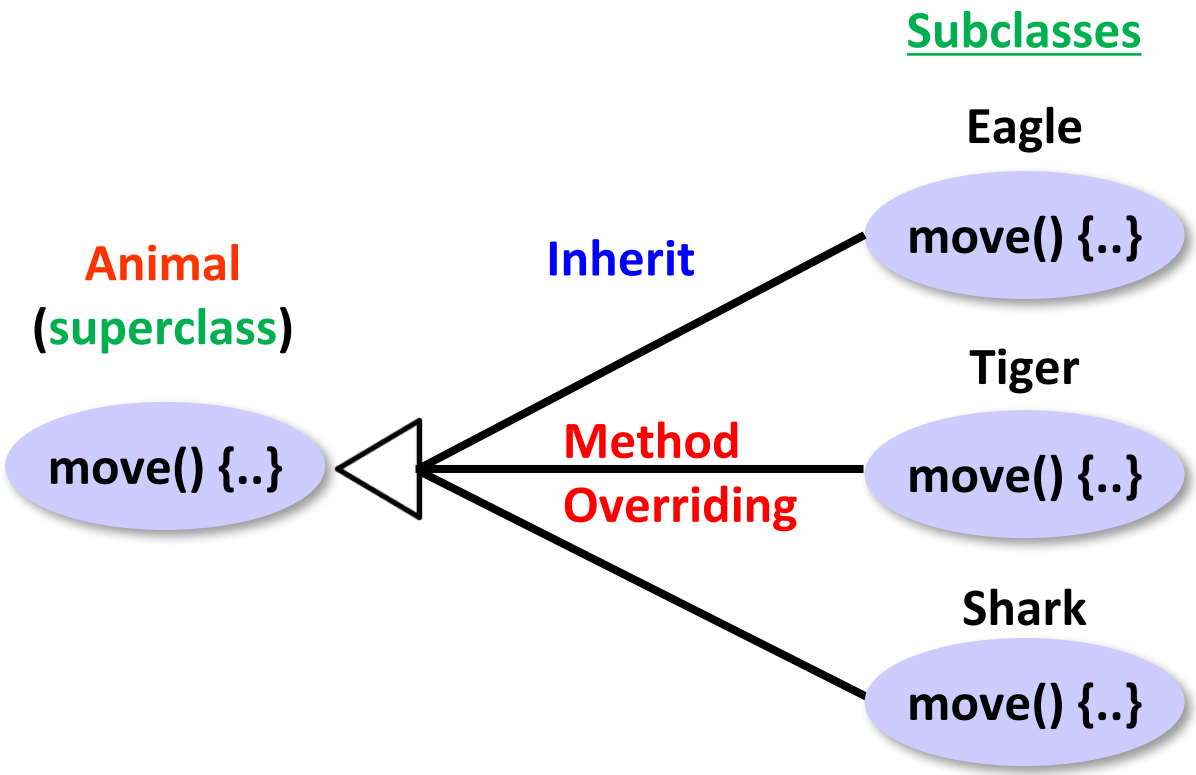
- Polymorphism means “many forms” (in Greek).
- In OOP, it means the ability of an object reference being referred to different types; knowing which method to apply depends on where it is in the inheritance hierarchy.
- Overriding - a necessary tool for polymorphism.
- A subclass can override a method in the parent class by defining a method with exactly the same signature and return type.
- The subclass method can replace or refine the method in the parent class.

Example

```
public class Person{
   int age ;
   String name;
// usual constructor, accessor mutator methods
…..
   public void printInfo() {
       System.out.println("Name is " + name);
       System.out.println("Age is " + age);
   }
   ……..
}
public class Employee extends Person{
   double salary ;
// usual constructor, accessor mutator methods
…….
   public void printInfo() {  // refine printInfo method
      System.out.println("Employee name is :" + getName()+
                         " and age is " + age );
      System.out.println("Salary is " + salary) ;
   }
}                  // replace printInfo method
```

method overriding

Polymorphism




```
Animal animal = new Eagle();
animal.move();

animal = new Tiger();
animal.move();

animal = new Shark();
animal.move();
```

Examples of Polymorphism

- Polymorphism:
  - When a program invokes a method through a superclass variable, the correct subclass version of the method is called, based on the type of the reference stored in the superclass variable.
    object
  - The same method name and signature can cause different actions to occur, depending on the type of object on which the method is invoked.
  - Facilitates adding new classes to a system with minimal modifications to the system's code.

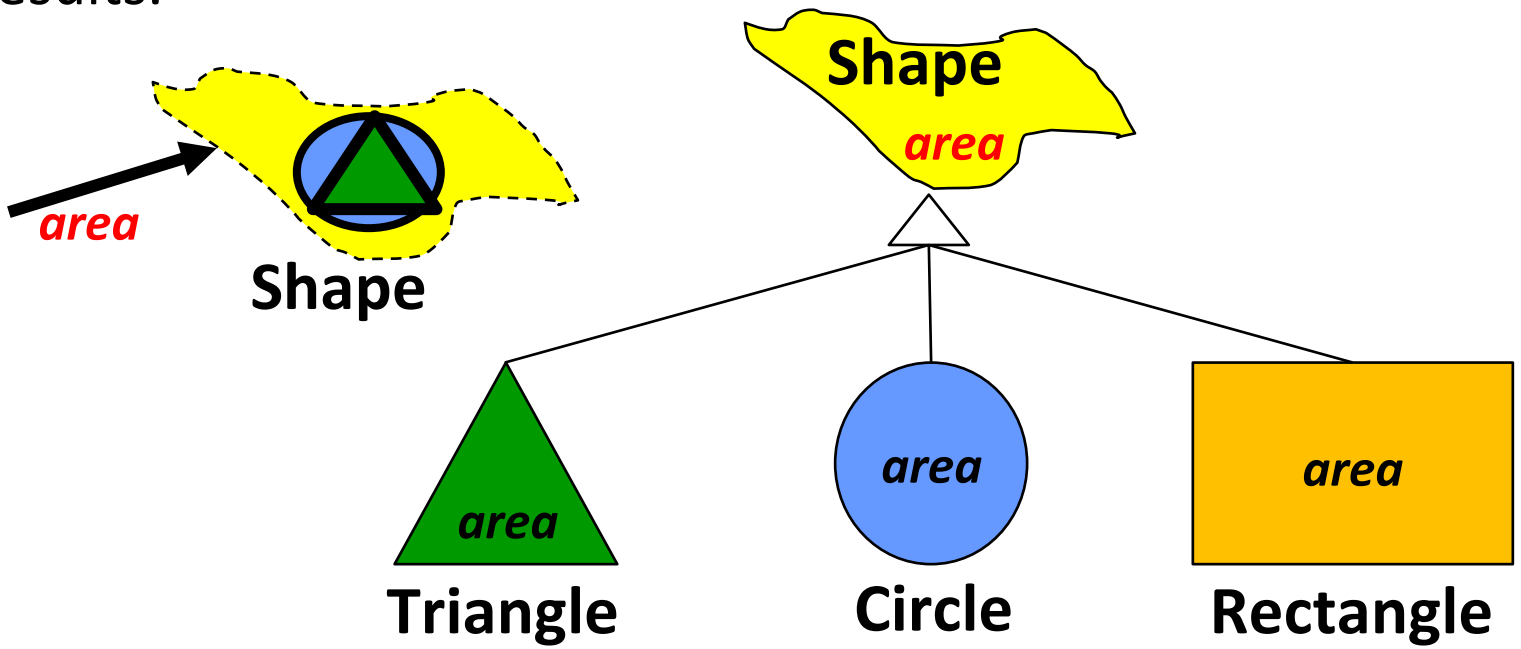
```
void  call (Animal animal)
{
        animal.move();
}
```

```
call(new Eagle());
animal = new Eagle();

call(new Tiger());
animal = new Tiger();

call(new Shark());
animal = new Shark();
```

Examples of Polymorphism

- For example, given a base class *shape,* polymorphism enables the programmer to define different *area* methods for any number of derived classes, such as *circles, rectangles* and *triangles.*

- No matter what shape an object is, applying the area method to it will return the correct results.



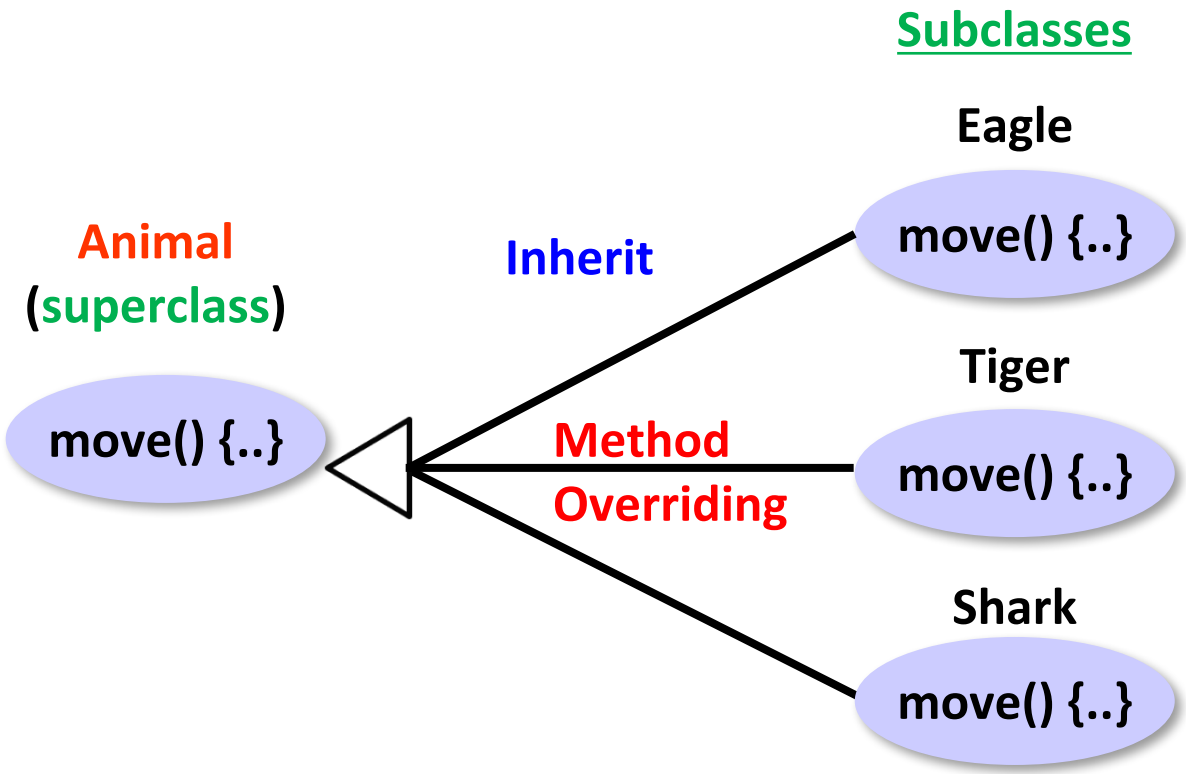
The Liskov Substitution Principle

- Barbara Liskov first wrote in 1988:

  "If for each object o_1 of type **S** there is an object o_2 of type **T** such that for all programs **P** defined in terms of **T**, the behavior of **P** is unchanged when o_1 is substituted for o_2 then **S** is a subtype of **T**."

- Paraphrased
  - Subtypes must be substitutable for their base types.

Polymorphism



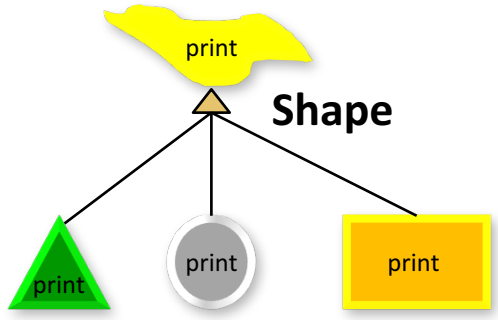

```
void  call (Animal animal)
{
        animal.move();
}
```

```
call(new Eagle());
animal = new Eagle();

call(new Tiger());
animal = new Tiger();

call(new Shark());
animal = new Shark();
```

Topic 2: Binding

Binding

- Binding - Refers which method to be called at a given time [i.e. connecting a **method call** to a ***method body***],
  circle.print();
  print() { …//implementation……}



- Static binding (early binding)
  - It occurs when the method call is "bound" at compile time

Shape circle = new Circle();

- Dynamic binding (late binding)
  - The selection of method body to be executed is delayed until execution time (based on the **actual object** referred by the reference) - (*execution time = runtime)*

Binding

- Java uses dynamic binding (by default) which enables the actual type of the object (object type), not the declared type (reference type), to govern which method to use.

Class_Name Object_Variable_Name = new *Class_Name*()
Interface_Name Object_Variable_Name = new *Class_Name*()
<reference type> <reference name> = new <Object Type> ()
<declared type> <object reference>
<variable type>

Example: Object anObj = new Person()


Example of Static Binding

```
public class StaticBindingTest {

    public static void main(String args[])  {
       Collection c = new HashSet();
       StaticBindingTest et = new StaticBindingTest();
       et.sort(c);

    }

    //overloaded method takes Collection argument
    public Collection sort(Collection c){
        System.out.println("Inside Collection sort method");
        return c;
    }


//another overloaded method which takes HashSet argument which is sub class
    public Collection sort(HashSet hs){
        System.out.println("Inside HashSet sort method");
        return hs;
    }

}
```

Example of Dynamic Binding

```
public class DynamicBindingTest {

    public static void main(String args[]) {
        Vehicle vehicle = new Car(); // here Type is vehicle but object will
                    // be Car
        vehicle.start();  //Car's start called because start() is overridden
             // method
    }
}
```

```
class Vehicle {
    public void start() {
        System.out.println("Inside start method of Vehicle");
    }
}
```

```
class Car extends Vehicle {
    // Override
    public void start() {
        System.out.println("Inside start method of Car");
    }
}
```

```
class Bus extends Vehicle {
    // Override
    public void start() {
        System.out.println("Inside start method of Bus");
    }
}
```

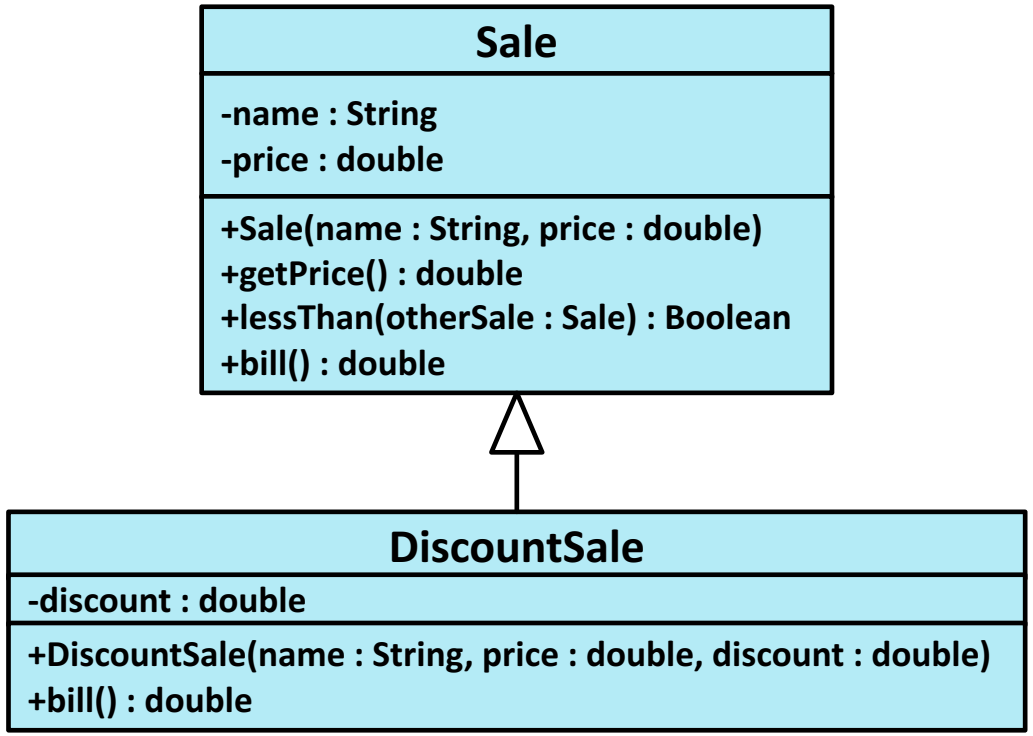
```
Output: Inside start method of Car
```

Dynamic Binding

- Java uses dynamic binding for all methods (except private, final, and static methods).
- Because of dynamic binding, a method can be written in a base class to perform a task, even if portions of that task aren't yet defined.

*'Program in the **general**' rather than 'program in **specific.**'*

- For an example, the relationship between a base class called *Sale* and its subclass *DiscountSale* will be examined.



Dynamic Binding

```
public class Sale {
……
public double bill( ){
   return price;
}
public boolean lessThan (Sale otherSale){
  if (otherSale == null)
  {
    System.out.println("Error: null
                        object");
    System.exit(0);
  }
  return (bill( ) < otherSale.bill( ));
}
}
```

```
public class DiscountSale extends Sale {
    …….
    public double bill( ) {
        ……???
    }
}
```

```
Sale simple = new sale("floor mat", 10.00);
DiscountSale discount = new DiscountSale("floor mat", 11.00, 10);
  . . .
if (!simple.lessThan(discount))
  System.out.println("code can still be compiled!");
```

Topic 3: Object Typecasting

Object Typecasting

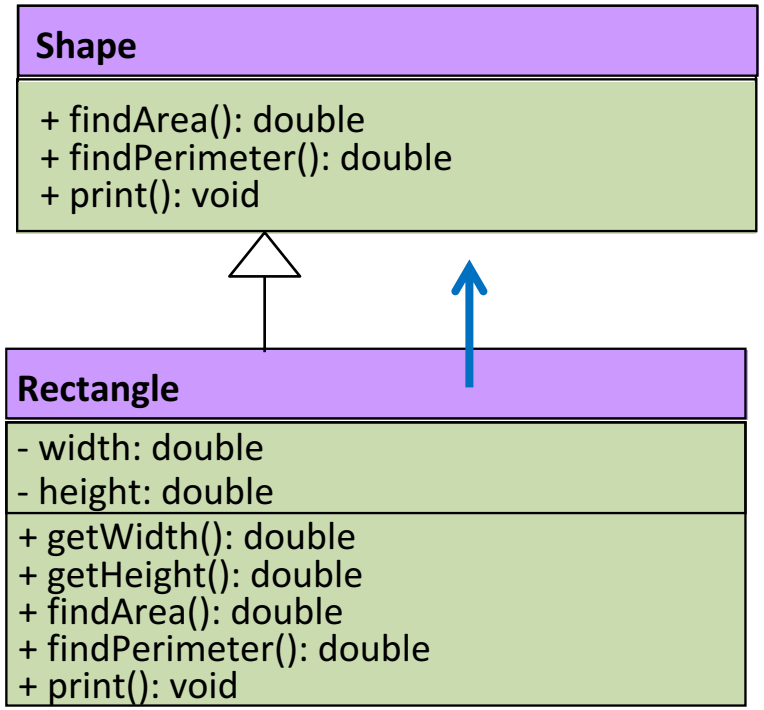
- **Upcasting** is when an object of a derived class is assigned to a variable of a base class (or any ancestor class).

```
Shape shape; //Base class
Rectangle rect= new Rectangle(15,10); //Derived class
shape = rect ; //Upcasting
```

```
System.out.println(shape.findArea());
```

- Because of dynamic binding, findArea() above uses the definition given in the Rectangle class.

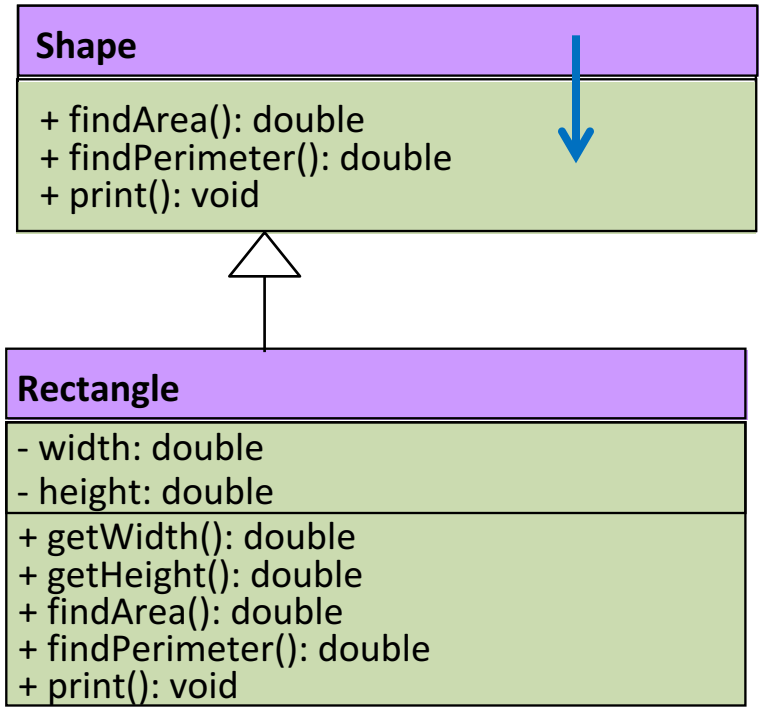
```
Shape shape = new Rectangle(15,10));
```



Upcasting and Downcasting

- **Downcasting** is when a type cast is performed from a base class to a derived class (or from any ancestor class to any descendent class).
  - Downcasting has to be done very carefully
  - In many cases it doesn't make sense, or is illegal:

```
Shape shape = new Shape() ;

Rectangle rect = shape ;
// will have compilation error

Rectangle  rect = (Rectangle)shape ;
// explicit cast, accept by compiler
//will produce runtime error !!!!

int height = rect.getHeight();
// compiler OK
```



Upcasting and Downcasting

- It is the responsibility of the programmer to use downcasting only in situations where it makes sense.
- The compiler does not check to see if downcasting is a reasonable thing to do.
- Using downcasting in a situation that does not make sense usually results in a run-time error.

Checking to See if Downcasting is Legitimate

- Downcasting to a specific type is only sensible if the object being cast is an instance of that type

```
Shape shape = new Rectangle();
Rectangle rect = (Rectangle) shape;
```

- This is exactly what the instanceof operator tests for:

```
object instanceof ClassName
```

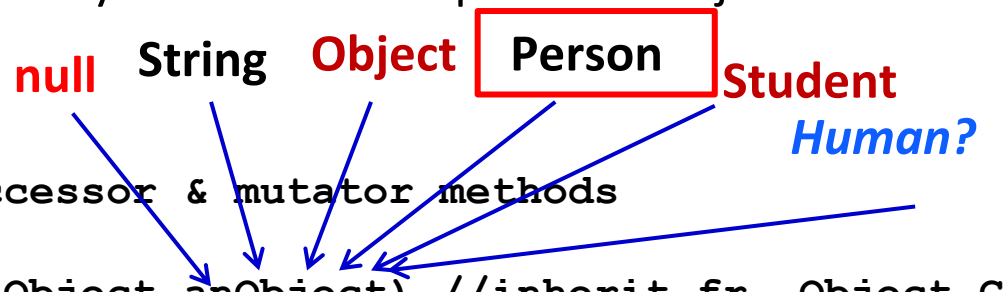
- It will return true if *object* is of type *ClassName*

- In particular, it will return true if *object* is an instance of any **descendent** class of *ClassName*

```
personObj instanceof Object
```

Use of Downcasting

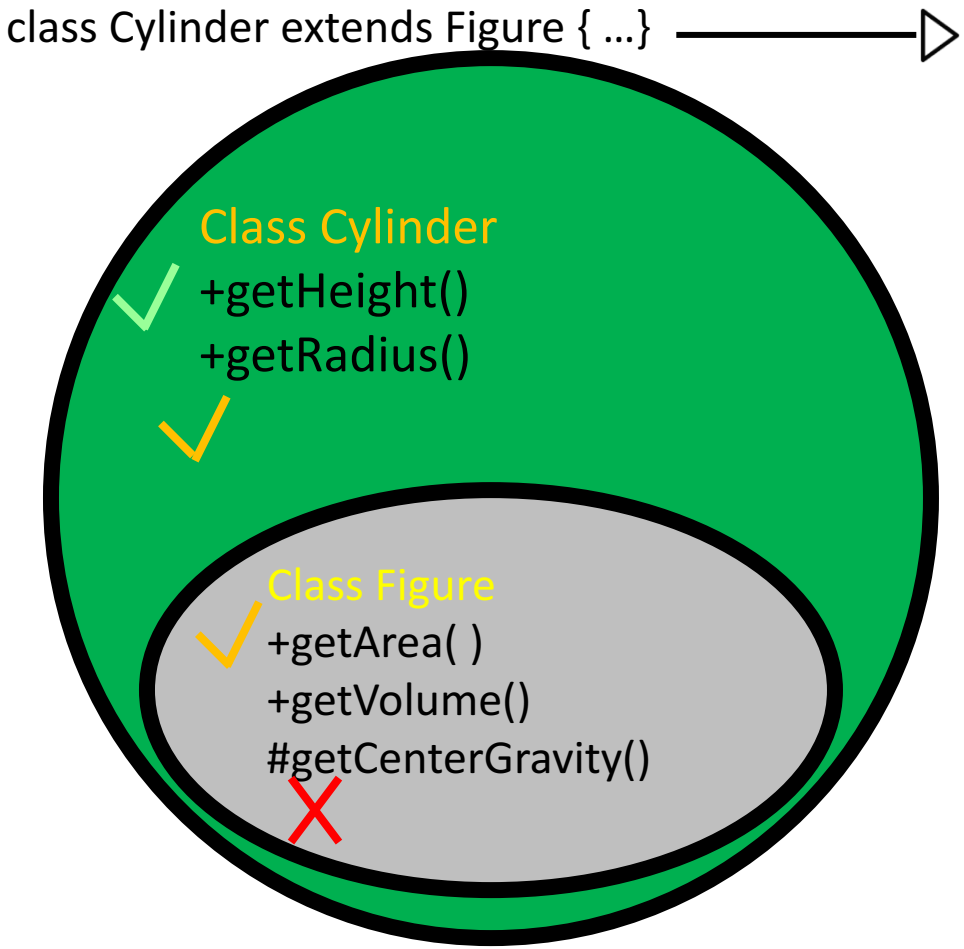
- **Downcasting** is very useful when you need to compare one object to another:




```
public class Person {
   private String name;
   private int age;
// usual constructor, accessor & mutator methods
…
   public boolean equals(Object anObject) //inherit fr. Object Class
   {
      if (anObject == null)
          return false;
     else if (!(anObject instanceof Person))
      /* else if (getClass( ) != anObject.getClass()) */
          return false;
      else
      {
         Person aPerson = (Person) anObject; // downcast
         return ( name.equals(aPerson.getName())
                  && (age == aPerson.getAge()));
      }
   }
}
```

Object anObject = new String("Tom");
Person aPerson = (Person)anObject ;

Use of Downcasting



Upcast

```
Figure fig = new Cylinder() ;
fig.getArea() ;
// compile & runtime both OK
```

Downcast

```
Figure fig = new Figure()
Cylinder cy = fig;  // compile NOK

Cylinder cy = (Cylinder) fig;
// explicit cast, compile OK,
// runtime NOK
cy.getHeight() ; // compile OK
```

<reference type> <reference name> = new <Object Type> ()

use at compile time to check

use at runtime to check

Allowed Assignments Between Superclass and Subclass Variables

- Superclass and subclass assignment rules
  - Assigning a superclass reference to a superclass variable is straightforward **Object ob = new Object()**
  - Assigning a subclass reference to a subclass variable is straightforward **String st = new String()**
  - Assigning a subclass reference to a superclass variable is safe because of the *is-a* relationship **Object st = new String()**
    - Referring to subclass-only members through superclass variables is a compilation error **st.charAt(0)**
  - Assigning a superclass reference to a subclass variable is a compilation error **String ob = new Object()**
    - Explicit downcasting can get around this error **String ob = (String)new Object()**

Topic 4: Benefits of Polymorphism

Benefits of Polymorphism

- Simplicity
  - If you need to write code that deals with a family of types, the code can ignore type-specific details and just interact with the base type of the family.
  - Even though the code thinks it is using an object of the base class, the object's class could actually be the base class or any one of its subclasses.
  - This makes your code easier for you to write and easier for others to understand.

Benefits of Polymorphism

- Extensibility
  - With polymorphism, programs become easily extensible.
  - We can add new functionality by creating new classes (or data types) inherited from an off-the-shelf base class without modifying the base class and the other classes derived from the base class.

Benefits of Polymorphism

```
public abstract class Shape {
        public abstract double area() ;
}
```

```
public class Rectangle extends Shape {
        private double width, height;
        public Rectangle(double w, double h)  { width = w ; height = h ; }
        public double area()  { return width * height ; }
}
```

```
public class Circle extends Shape {
        private double radius;
        public Rectangle(double r)  { radius = r ;  }
        public double area()  { return  Math.PI * radius * radius ; }
}
```

```
public class ShapeApp {
       public static void main(String[ ] args) {
shapes[1] = new Circle(8) ;
              for (Shape s : shapes) { System.out.println("Area  = " + s.area() ; }
      ArrayList<Circle> shapes = new ArrayList<Circle>() ;
      shapes.add(new Circle(0.1));
      shapes.add(new Circle(0.8));
```

Topic 5: Three Ways of Method Overriding

Three Ways of Method Overriding

- Method overriding: Methods of a subclass override the methods of a superclass.
- Method overriding (implementation) of the abstract methods: Methods of a subclass implement the abstract methods of an abstract class.
- Method overriding (implementation) through the Java interface: Methods of a concrete class implement the methods of the interface.

Topic 6: More Examples

Example

```
interface IAnimal {
        void move();
        void speak();
}
class Cat implements IAnimal{
    public void move() {
        System.out.println("Cat
        moves....");
  }
    public void speak() {
        System.out.println("Meow !"); }
}
class Lion implements IAnimal {
    public void move() {
        System.out.println("Lion
        moves...");}
    public void speak() {
        System.out.println("ROAR !"); }
}
```

```
class CareTaker
{
  public void takeAWalk(IAnimal pet)
    {       pet.move();
            pet.speak();
     }

}

class  AnyClass {
…….
    CareTaker ct = new CareTaker();
        Cat cat = new Cat();
        Lion lion = new Lion();

        ct.takeAWalk(cat);
        ct.takeAWalk(lion);
……..
}
```

Example

```
interface IAnimal {
        void move();
        void spea
}
class Cat implem
    public void m
        System.o
        moves....'
  }
    public void sp
        System.out.println("Meow !"); }
}
class Lion implements IAnimal {
```

Method Overloading.
Which method to be called is known during compile time
=>
Static Binding

```
class CareTaker
{
public void takeAWalk (Cat cat)  {....}
public void takeAWalk (Lion lion) {.....}
public void takeAWalk (.........) {........}

}

class  AnyClass {
...............
    CareTaker ct = new
        Cat cat = new
        Lion lion = new

        ct.takeAWalk(
        ct.takeAWalk(
...................
}
```

Only during runtime, will the object implementing IAnimal be known
=>
Dynamic Binding

Dog implements IAnimal
Elephant implements IAnimal
Tiger implements IAnimal
.....

Example

```
abstract class LivingThings {
        public  void grow( ) {
                System.out.println("LivingThings
                grow!"); }
       public abstract void speak( );
  }
  abstract class Mammal extends LivingThings {
      public void move( ) {
                System.out.println("Mammal move!"); }
        static
      public void grow()  {
                System.out.println("Mammal  grow!"); }

      public void eat()  {
                System.out.println("Mammal eat!"); }
      }
   class Dog extends Mammal {  // concrete class
      public void speak( ) {
                System.out.println("Woof!"); }
      public  void move( ) {
      static System.out.println("Dog move!"); }
      public void grow( ) {
                System.out.println("Dog grow!"); }
   }
```

```
class House {
    public static void main(
            String[] args) {
        Dog mDog = new Dog( );
        mDog.speak( );
        mDog.move( );
        mDog.eat();
        mDog.grow();

        Mammal m = new Dog( );
        m.speak( );
        m.move( );
        m.eat();
        m.grow();


    }
}

}
```

Example

```
abstract class LivingThings {
        public  void grow( ) {
                System.out.println("LivingThings
                grow!"); }
       public abstract void speak( );
  }
  abstract class Mammal extends LivingThings {
      public void move( ) {
                System.out.println("Mammal move!"); }
        static
      public void grow()  {
                System.out.println("Mamm
      public void eat()  {
                System.out.println("Mammal eat!"); }
      }
   class Dog extends Mammal {  // concrete class
      public void speak( ) {
                System.out.println("Woof!"); }
      public  void move( ) {
      static    System.out.println("Dog move!"); }
      public void grow( ) {
                System.out.println("Dog grow!"); }
   }
```

```
class House {
    public static void main(
          String[] args) {
       Dog mDog = new Dog( );
       mDog.speak( );
                                  = new Dog( );
       m.eat();
       m.grow();
```

abstract, subclass must 'implement'

Mammal did not implement abstract method speak so has to be abstract

Dog inherit eat method
And overrides move & grow methods
And implements abstract method speak

Program Output

```
Woof!
Dog moves!
Mammal eats!
Dog grows!

Woof!
Dog moves!
Mammal eats!
Dog grows!
```

Summary

Key points from this chapter:

- Polymorphism means “many forms” (in Greek). In OOP it means the ability of an object reference being referred to different types; knowing which method to apply depends on where it is in the inheritance hierarchy.
- The Liskov Substitution Principle states that subtypes must be substitutable for their base types.
- The benefits of polymorphism is simplicity and extensibility.
- It is the responsibility of the programmer to use downcasting only in situations where it makes sense; otherwise results in a run-time error.